

ZAAWANSOWANE PROGRAMOWANIE OBIEKTOWE

Sprawozdanie z Laboratorium nr 4

Implementacja usługi sieciowej typu REST

Szymon Banaszek

Data: 03.06.2026

1. Cel cwiczenia

Celem laboratorium było zaprojektowanie i zaimplementowanie usługi sieciowej typu REST przy użyciu frameworka Spring Boot. Cwiczenie obejmowało konfigurację połączenia z bazą danych PostgreSQL, implementację warstwy encji (JPA), repozytoriów, serwisów oraz kontrolerów REST. Zrealizowano także mechanizm uwierzytelniania Basic Auth za pomocą Spring Security, walidację danych wejściowych oraz automatyczną dokumentację API z wykorzystaniem biblioteki OpenAPI (Swagger).

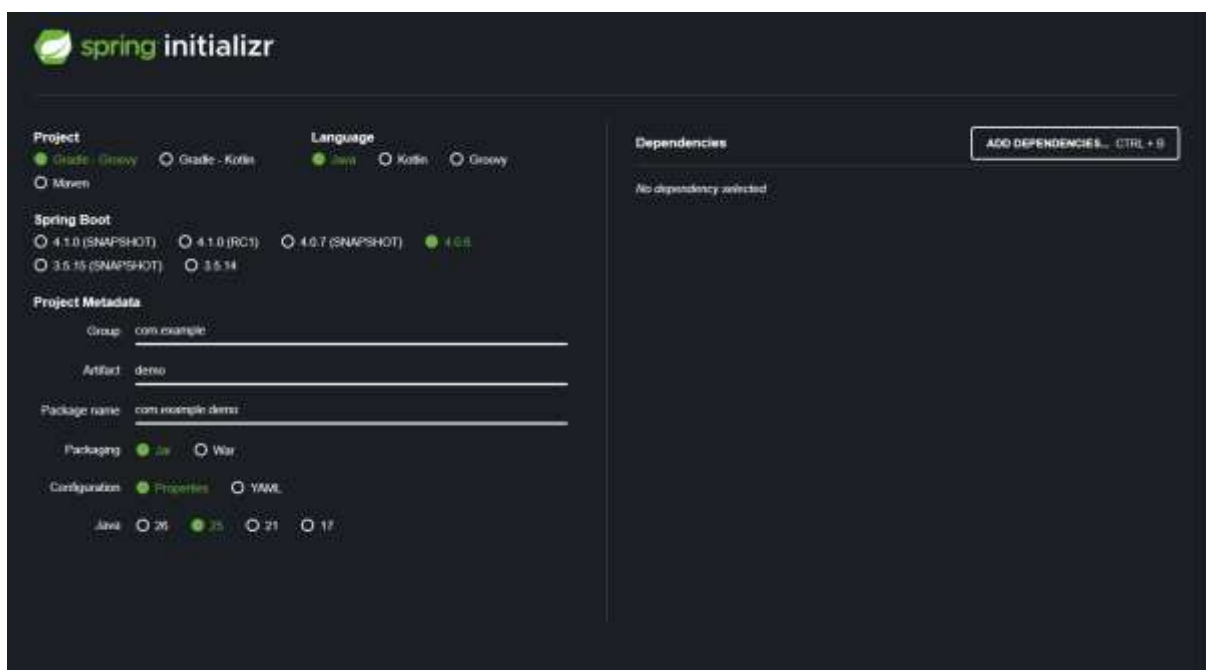
2. Środowisko i technologie

Do realizacji laboratorium wykorzystano następujące narzędzia i technologie:

- Java 25.0.1 – język programowania
- Spring Boot 4.0.6 – framework aplikacyjny
- Spring Data JPA / Hibernate – warstwa persystencji
- Spring Security – uwierzytelnianie Basic Auth
- PostgreSQL 18 – relacyjna baza danych
- Lombok – automatyczne generowanie kodu powtarzalnego
- springdoc-openapi 2.8.9 (Swagger UI) – dokumentacja REST API
- IntelliJ IDEA 2026.1 – środowisko programistyczne
- Gradle – system budowania projektu

3. Generowanie projektu

Szkielet projektu został wygenerowany za pomocą narzędzia Spring Initializr (<https://start.spring.io>). Wybrano następujące ustawienia: Project: Gradle-Groovy, Language: Java, Group: com.project, Artifact: project-rest-api. Dodano zależności: Spring Web, Spring Security, Validation, Spring Data JPA, PostgreSQL Driver, Lombok, Spring Boot DevTools.



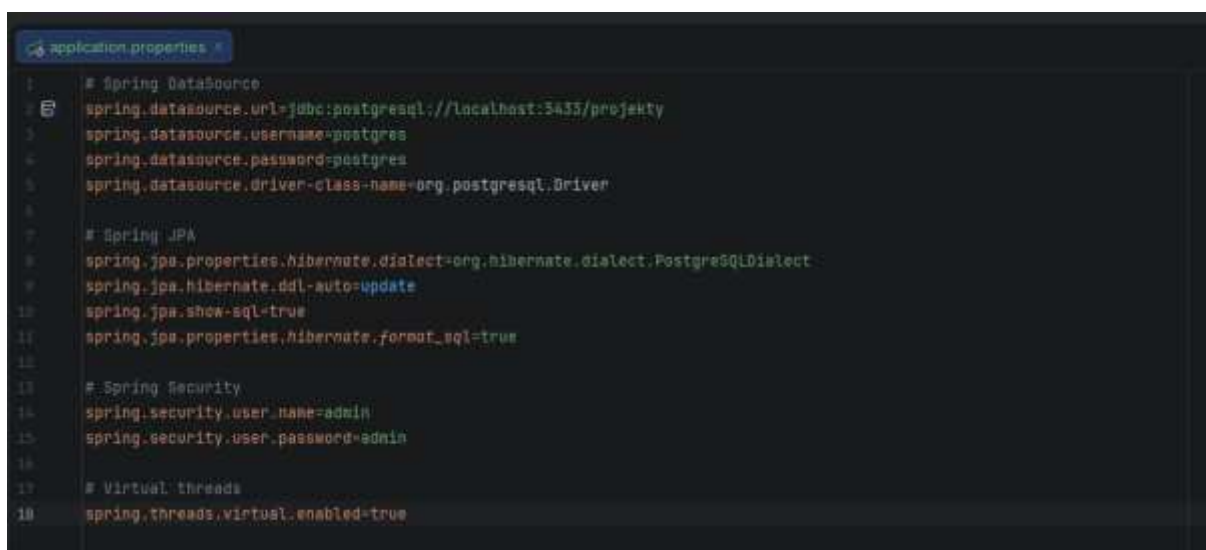
The screenshot shows the Spring Initializr web interface. The 'Project' section has 'Gradle - Groovy' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '4.0.6' selected. The 'Project Metadata' section shows 'Group' as 'com.example', 'Artifact' as 'demo', and 'Package name' as 'com.example.demo'. The 'Packaging' section has 'Jar' selected. The 'Configurations' section has 'Properties' selected. The 'Java' version is set to '25'. The 'Dependencies' section is empty, with a button to 'ADD DEPENDENCIES... CTRL + S'.

Wygenerowane archiwum ZIP zostało rozpakowane do folderu projektu i zaimportowane do IntelliJ IDEA jako projekt Gradle. Napotkano problem z niezgodnością wersji Java (toolchain wymagał Java 21, zainstalowana była Java 25) – rozwiązano przez zmianę wersji toolchain w pliku build.gradle.

4. Konfiguracja aplikacji

4.1. Plik application.properties

W pliku konfiguracyjnym zdefiniowano parametry połączenia z bazą danych PostgreSQL, ustawienia Hibernate oraz dane uwierzytelniające Spring Security. Włączono również mechanizm wątków wirtualnych (Virtual Threads – `spring.threads.virtual.enabled=true`), który pozwala na obsługę znacznie większej liczby jednoczesnych zadań HTTP bez ryzyka blokady zasobów serwera.



```
1 # Spring DataSource
2 spring.datasource.url=jdbc:postgresql://localhost:5435/projekty
3 spring.datasource.username=postgres
4 spring.datasource.password=postgres
5 spring.datasource.driver-class-name=org.postgresql.Driver
6
7 # Spring JPA
8 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
9 spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11 spring.jpa.properties.hibernate.format_sql=true
12
13 # Spring Security
14 spring.security.user.name=admin
15 spring.security.user.password=admin
16
17 # Virtual threads
18 spring.threads.virtual.enabled=true
```

4.2. Konfiguracja Spring Security – klasa SecurityConfig

Klasa SecurityConfig oznaczona adnotacją @Configuration definiuje reguły zabezpieczeń aplikacji. Wyłączono ochronę CSRF (niezbędna dla REST API testowanego przez Postmana/Swagger), ustawiono wymóg uwierzytelnienia dla każdego zadania oraz skonfigurowano mechanizm Basic Auth. Adnotacja @Bean informuje Spring, że ma zarządzać obiektem zwracanym przez metodę filterChain.

```

package com.project.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
        return httpSecurity
            .csrf(Customizer.withDefaults())
            .authorizeHttpRequests((AuthorizationManagerRequestMatcher) auth -> auth.anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults())
            .build();
    }
}

```

4.3. Konfiguracja JPA Auditing – klasa JpaConfig

Klasa JpaConfig z adnotacją @EnableJpaAuditing włącza mechanizm automatycznego uzupełniania pól @CreatedDate i @LastModifiedDate w encjach. Dzięki temu Spring automatycznie ustawia daty przy tworzeniu i modyfikacji rekordów w bazie danych, bez potrzeby ręcznego ustawiania tych wartości.

5. Implementacja warstw aplikacji

5.1. Warstwa modelu – encje JPA

Zaimplementowano trzy klasy encyjne odwzorowujące tabele w bazie danych: Projekt, Zadanie oraz Student. Klasy oznaczono adnotacją @Entity oraz @Table. Zastosowano adnotacje biblioteki Lombok (@Getter, @Setter, @Builder, @NoArgsConstructor, @AllArgsConstructor), które automatycznie generują gettery, settery i konstruktory podczas kompilacji, eliminując zbędny powtarzalny kod.

Zaimplementowane relacje między encjami:

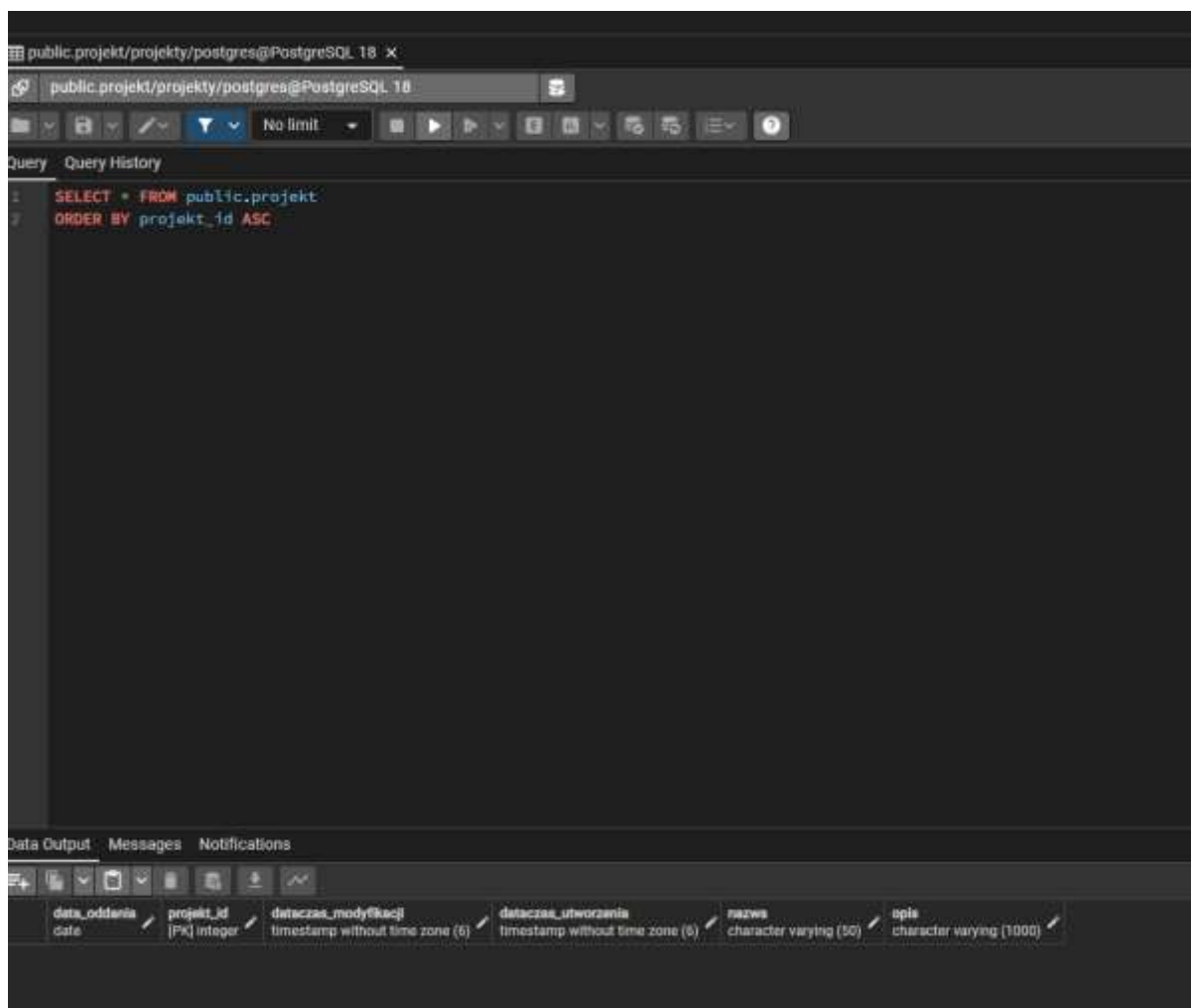
- Projekt – Zadanie: relacja jeden-do-wielu (@OneToMany / @ManyToOne). Właścicielem relacji jest Zadanie (posiada @JoinColumn z kluczem obcym projekt_id).
- Projekt – Student: relacja wiele-do-wielu (@ManyToMany) z tabelą pośrednią projekt_student zdefiniowaną przez @JoinTable po stronie klasy Projekt.

Aby uniknąć nieskończonej rekurencji podczas serializacji do JSON (zapętlenie), zastosowano adnotację @JsonIgnoreProperties po stronie kolekcji @OneToMany.

```

1 package com.project.model;
2
3 import jakarta.persistence.*;
4
5
6
7
8
9
10
11
12
13
14
15
16 @Getter
17 @Setter
18 @Builder
19 @NoArgsConstructor
20 @AllArgsConstructor
21 @EntityListeners(AuditingEntityListener.class)
22 @Entity
23 @Table(name = "projekt")
24 public class Projekt {
25
26     @Id
27     @GeneratedValue
28     @Column(name = "projekt_id")
29     private Integer projektId;
30
31     @NotBlank(message = "Pole nazwa nie może być puste!")
32     @Size(min = 1, max = 50, message = "Nazwa musi zawierać od 1 do 50 znaków!")
33     @Column(nullable = false, length = 50)
34     private String nazwa;
35
36     @Column(length = 1000)
37     private String opis;
38
39     @CreatedDate
40     @Column(name = "data_czas_ustalenia", nullable = false, updatable = false)
41     private LocalDateTime dataCzasUstalenia;
42
43     @LastModifiedDate
44     @Column(name = "data_czas_modyfikacji", insertable = false)
45     private LocalDateTime dataCzasModyfikacji;
46
47     @Column(name = "data_zadania")
48     private LocalDate dataZadania;
49
50     @OneToMany(mappedBy = "projekt")
51     @JsonIgnoreProperties({"projekt"})
52     private List<Zadanie> zadania;
53
54     @ManyToMany
55     @JoinTable(name = "projekt_student",
56         joinColumns = {@JoinColumn(name = "projekt_id")},
57         inverseJoinColumns = {@JoinColumn(name = "student_id")})
58     private Set<Student> studenci;
59 }

```



5.2. Warstwa repozytoriów

Repozytoria to interfejsy dziedziczące z `JpaRepository<T, ID>`, które zapewniają dostęp do podstawowych operacji bazodanowych bez potrzeby ich implementacji. Zdefiniowano trzy repozytoria: `ProjektRepository`, `ZadanieRepository` oraz `StudentRepository`.

Zastosowano dwa mechanizmy definiowania zapytań:

- Derived queries – Spring automatycznie generuje SQL na podstawie nazwy metody, np. `findByNazwaContainingIgnoreCase` -> `SELECT p FROM Projekt p WHERE upper(p.nazwa) LIKE upper('%:nazwa%')`
- Annotacja `@Query` – ręczne zapytania JPQL, używane gdy nazwa derived query byłaby zbyt długa lub dla lepszej czytelności, np. w `ZadanieRepository`.

```

ProjektRepository.java
package com.project.repository;

import com.project.model.Projekt;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface ProjektRepository extends JpaRepository<Projekt, Integer> {
    Page<Projekt> findByNazwaContainingIgnoreCase(String nazwa, Pageable pageable);
    List<Projekt> findByNazwaContainingIgnoreCase(String nazwa);
}

ZadanieRepository.java
package com.project.repository;

import ...

public interface ZadanieRepository extends JpaRepository<Zadanie, Integer> {
    @Query("SELECT z FROM Zadanie z WHERE z.projekt.projektId = :projektId")
    Page<Zadanie> findZadaniaProjektu(@Param("projektId") Integer projektId, Pageable pageable);

    @Query("SELECT z FROM Zadanie z WHERE z.projekt.projektId = :projektId")
    List<Zadanie> findZadaniaProjektu(@Param("projektId") Integer projektId);
}

```

5.3. Warstwa serwisów

Serwisy stanowią warstwę logiki biznesowej pośredniczącą między kontrolerami a repozytoriami. Każdy serwis został zdefiniowany jako interfejs oraz klasa implementacyjna oznaczona adnotacją `@Service`. Wstrzykiwanie zależności odbywa się przez konstruktor.

W metodzie `deleteProjekt` zastosowano adnotację `@Transactional`, która gwarantuje że usunięcie zadań i projektu odbywa się w jednej transakcji bazodanowej. Jeśli któraś operacja się nie powiedzie, całość zostaje wycofana (rollback). Jest to równoważne ręcznemu zarządzaniu transakcją `EntityTransaction`.

```

package com.project.service;

import com.project.model.Projekt;
import com.project.repository.ProjektRepository;
import com.project.repository.ZadanieRepository;
import jakarta.transaction.Transactional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class ProjektServiceImpl implements ProjektService {

    private ProjektRepository projektRepository;
    private ZadanieRepository zadanieRepository;

    @Autowired
    public ProjektServiceImpl(ProjektRepository projektRepository, ZadanieRepository zadanieRepository) {
        this.projektRepository = projektRepository;
        this.zadanieRepository = zadanieRepository;
    }

    @Override
    public Optional<Projekt> getProjekt(Integer projectId) { return projektRepository.findById(projectId); }

    @Override
    public Projekt setProjekt(Projekt projekt) { return projektRepository.save(projekt); }

    @Override
    @Transactional
    public void deleteProjekt(Integer projectId) {
        zadanieRepository.deleteAll(zadanieRepository.findAllByProjektId(projectId));
        projektRepository.deleteById(projectId);
    }

    @Override
    public Page<Projekt> getProjekty(Pageable pageable) { return projektRepository.findAll(pageable); }

    @Override
    public Page<Projekt> searchByNazwa(String nazwa, Pageable pageable) {
        return projektRepository.findAllByNazwaContainingIgnoreCase(nazwa, pageable);
    }
}

```

5.4. Warstwa kontrolerow REST

Zaimplementowano trzy kontrolery REST. Klasy oznaczono adnotacją `@RestController` oraz `@RequestMapping("/api")`. Każdy endpoint korzysta z adnotacji HTTP (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`) oraz parametrów `@PathVariable`, `@RequestParam` i `@RequestBody`.

Operacja POST zwraca kod 201 Created z nagłówkiem Location wskazującym na URL nowoutworzonego zasobu. Operacje PUT i DELETE sprawdzają czy zasób istnieje i zwracają 404 Not Found jeśli nie.

Walidacja danych wejściowych realizowana jest przez adnotacje `@Valid` w parametrach metod kontrolera, co włącza sprawdzanie ograniczeń zdefiniowanych w modelu (`@NotBlank`, `@Size`).


```

1 import java.net.URI;
2
3 @RestController
4 @RequestMapping("/api")
5 @Tag(name = "Projekt")
6 public class ProjektRestController {
7
8     private ProjektService projektService;
9
10    @Autowired
11    public ProjektRestController(ProjektService projektService) {
12        this.projektService = projektService;
13    }
14
15    @GetMapping("/{projektId}")
16    ResponseEntity<Projekt> getProjekt(@PathVariable("projektId") Integer projektId) {
17        return ResponseEntity.of(projektService.getProjekt(projektId));
18    }
19
20    @PostMapping("/{projektId}")
21    ResponseEntity<Void> createProjekt(@Valid @RequestBody Projekt projekt) {
22        Projekt created = projektService.setProjekt(projekt);
23        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
24            .path("/{projektId}").buildAndExpand(created.getProjectId()).toUri();
25        return ResponseEntity.created(location).build();
26    }
27
28    @PutMapping("/{projektId}")
29    ResponseEntity<Void> updateProjekt(@Valid @RequestBody Projekt projekt,
30        @PathVariable("projektId") Integer projektId) {
31        return projektService.getProjekt(projektId).flatMap(projekt -> {
32            projektService.setProjekt(projekt);
33            return new ResponseEntity<Void>(HttpStatus.OK);
34        }).orElse(ResponseEntity.notFound().build());
35    }
36
37    @DeleteMapping("/{projektId}")
38    ResponseEntity<Void> deleteProjekt(@PathVariable("projektId") Integer projektId) {
39        return projektService.getProjekt(projektId).flatMap(projekt -> {
40            projektService.deleteProjekt(projektId);
41            return new ResponseEntity<Void>(HttpStatus.OK);
42        }).orElse(ResponseEntity.notFound().build());
43    }
44
45    @GetMapping(value = "/{projektId}/pageable")
46    Page<Projekt> getProjekt(@PathVariable("projektId") Integer projektId,
47        @ParameterObject Pageable pageable) {
48        return projektService.getProjekt(projektId, pageable);
49    }
50
51    @GetMapping(value = "/{projektId}/nazwa/{nazwa}/pageable", params = {"nazwa"})
52    Page<Projekt> getProjektByNazwa(@PathVariable("projektId") Integer projektId,
53        @PathVariable("nazwa") String nazwa,
54        @ParameterObject Pageable pageable) {
55        return projektService.searchByNazwa(nazwa, pageable);
56    }
57 }

```

6. Dokumentacja API – Swagger UI

Do automatycznego generowania dokumentacji REST API dodano zależność `springdoc-openapi-starter-webmvc-ui:2.8.9` w pliku `build.gradle`. Po uruchomieniu aplikacji dokumentacja dostępna jest pod adresem `http://localhost:8080/swagger-ui/index.html`.

Interfejs Swagger UI prezentuje wszystkie endpointy pogrupowane według tagów (@Tag), umożliwia przeglądanie parametrów i schematów odpowiedzi oraz bezpośrednie testowanie API. Dzięki adnotacji `@ParameterObject` obiekt `Pageable` jest prawidłowo rozbity na parametry `page`, `size` i `sort`.

The image displays a REST client interface with three main sections: **Zadanie**, **Student**, and **Projekt**. Each section lists endpoints with their corresponding HTTP methods (GET, POST, DELETE).

Zadanie endpoints:

- GET /api/zadania/{zadanieId}
- POST /api/zadania/{zadanieId}
- DELETE /api/zadania/{zadanieId}
- GET /api/zadania
- POST /api/zadania

Student endpoints:

- GET /api/studenti/{studentId}
- POST /api/studenti/{studentId}
- DELETE /api/studenti/{studentId}
- GET /api/studenti
- POST /api/studenti

Projekt endpoints:

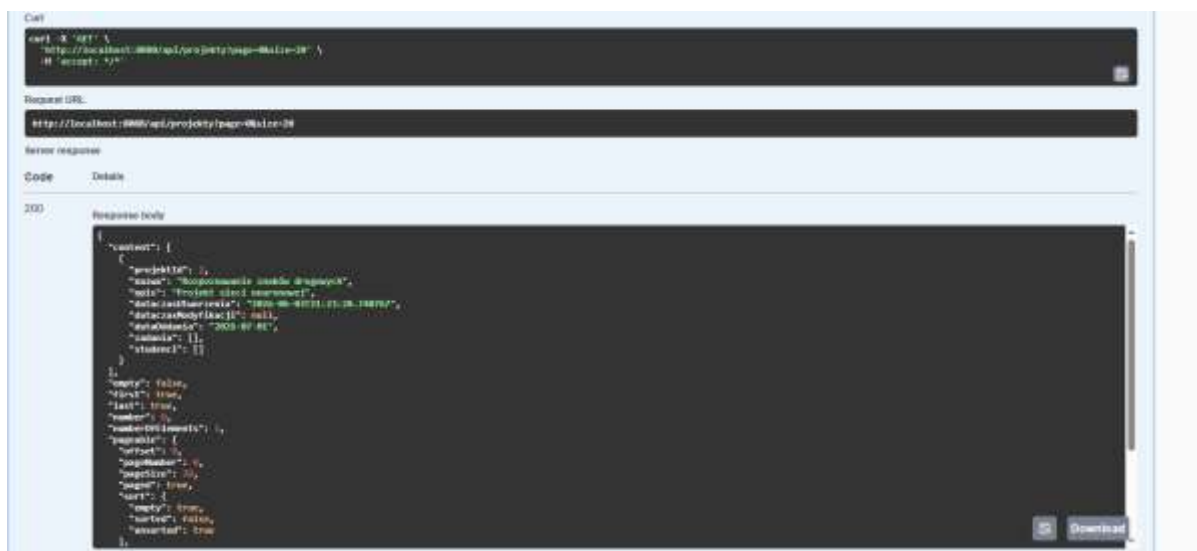
- GET /api/projekty/{projektId}
- POST /api/projekty/{projektId}
- DELETE /api/projekty/{projektId}
- GET /api/projekty
- POST /api/projekty
- GET /api/projekty-nazwa

The bottom section shows a detailed view of a **POST** request to **/api/projekty**. The request body is in **application/json** format:

```
{
  "nazwa": "Rozprawa o rzadzie drogowym",
  "opis": "Projekt sluzi rozprawom",
  "aktualizacja": "2020-01-01"
}
```

The interface includes buttons for **Execute** and **Clear**, and a **Responses** section at the bottom showing the raw response in a terminal-like window:

```
curl -X POST -H
'Host: localhost:8080/api/projekty' -H
'accept: */*' -H
'Content-Type: application/json' -d
'{"nazwa": "Rozprawa o rzadzie drogowym",
"opis": "Projekt sluzi rozprawom",
"aktualizacja": "2020-01-01"}'
}
```



7. Testy jednostkowe i integracyjne

Zaimplementowano testy dla kontrolera `ProjektRestController` oraz serwisu `ProjektServiceImpl`:

- `ProjektRestControllerTest` – test jednostkowy z Mockito. Nie uruchamia Spring, działa w milisekundach. Testuje logikę Java metod kontrolera, nie sprawdza adnotacji HTTP.
- `ProjektRestControllerIT` – test integracyjny warstwy webowej z `@WebMvcTest` i `MockMvc`. Uruchamia kontekst Spring Security, sprawdza pełną obsługę HTTP, walidację i serializację JSON.
- `ProjektServiceTest` – test jednostkowy serwisu weryfikujący logikę biznesową (save, delete z transakcją, wyszukiwanie).

Testy pisane byly zgodnie z wzorcem Given-When-Then. Zastosowano Mockito do tworzenia atrap (mockow) zaleznosci, co pozwala testowac klasy w izolacji.

Test Case	Duration
ProjektRestControllerIT (com.project.controller)	649 ms
createProjekt_whenEmptyName_returnsBadRequest()	411 ms
createProjekt_whenValidData_returnsCreatedWithLocation	65 ms
getProjekt_whenAvailable_returnsPagedContent()	106 ms
getProjekt_whenValidId_returnsProjekt()	23 ms
getProjekt_whenInvalidId_returnsNotFound()	16 ms
getProjekt_withPagingParams_passesParamsToService()	28 ms

Test Case	Duration
ProjektRestControllerTest (com.project.controller)	1 sec 73 ms
getProjekt_whenValidId_returnsProjekt()	951 ms
getProjekt_whenInvalidId_returnsNotFound()	2 ms
createProjekt_whenValidData_returnsCreated()	102 ms
deleteProjekt_whenValidId_returnsOk()	7 ms
getProjekt_returnsPageWithProjekt()	11 ms

8. Konfiguracja mechanizmu rejestracji

W pliku logback-spring.xml w katalogu src/main/resources skonfigurowano mechanizm rejestracji zdarzeń aplikacji. Logi są jednocześnie wypisywane na konsolę oraz zapisywane do plików w katalogu logs/. Zastosowano politykę rotacji dziennej z archiwizacją ZIP i maksymalną wielkością archiwum 30 MB.

Zamiast `System.out.println()` używane są metody SLF4J: `logger.info()`, `logger.warn()` i `logger.error()`. W klasach z adnotacją Lomboka `@Slf4j` zmienna `log` jest tworzona automatycznie, bez potrzeby wywołania `LoggerFactory.getLogger()`.

